

用户数据采集与 关联分析结课作业

数据可视化与智能分析

刘子康

第一讲

哔哩哔哩 (^- ^)つ口 干杯~ × | 动态首页-哔哩哔哩 × | 从来没想到在西班牙卖卤肉 × | python/ × | Untitled × | 003-NER-企业年报-数字技 × | THULAC: 一个高效的中文 ×

← ↻ 🏠 https://thulac.thunlp.org/demo

THULAC：一个高效的中文词法分析工具包

欢迎使用THULAC中文分词工具包demo系统

黄旭华，1926年3月12日出生于广东省汕尾市，原籍广东省揭阳市。1949年毕业于上海交通大学。历任北京海军核潜艇研究室副总工程师、中船重工集团公司核潜艇总体设计所研究员、名誉所长。1994年当选为中国工程院院士

【测试 Try】

黄旭华_np , _w 1926年_t 3月_t 12日_t 出生_v 于_p 广东省汕尾市_ns , _w 原籍_n 广东省_ns 揭阳市_ns 。 _w 1949年_t 毕业_v 于_p 上海交通大学_ni 。 _w 历任_v 北 京_ns 海军_n 核潜艇_n 研究室_n 副总_j 工程师_n 、 _w 中_f 船_n 重工_j 集团公 司_n 核潜艇_n 总体_n 研究_v 设计所_n 研究员_n 、 _w 名誉_n 所长_n 。 _w 1994年 _t 当选_v 为_v 中国_ns 工程院_n 院士_n

词性解释

n/名词 np/人名 ns/地名 ni/机构名 nz/其它专名
m/数词 q/量词 mq/数量词 t/时间词 f/方位词 s/处所词
v/动词 vm/能愿动词 vd/趋向动词 a/形容词 d/副词
h/前接成分 k/后接成分 i/习语 j/简称
r/代词 c/连词 p/介词 u/助词 y/语气助词
e/叹词 o/拟声词 g/语素 w/标点 x/其它

版权所有：清华大学自然语言处理与社会人文计算实验室
Copyright: Natural Language Processing and Computational Social Science Lab, Tsinghua University

1 基本操作

```
In [1]: !pip install jieba

Requirement already satisfied: jieba in d:\anaconda\lib\site-packages (0.42.1)

In [2]: import jieba # 在命令行里面安装分词软件包jieba # pip install jieba

In [3]: seg_list = jieba.cut("南京大学生都爱南京市长江大桥")

In [4]: print(' '.join(seg_list))

Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\HUAWEI\AppData\Local\Temp\jieba.cache
Loading model cost 1.000 seconds.
Prefix dict has been built successfully.

南京 大学生 都 爱 南京市 长江大桥

In [5]: seg_list = jieba.cut("南京大学生都爱南京市长江大桥")

In [6]: print(' * '.join(seg_list))

南京*大学生*都*爱*南京市*长江大桥
```

2 加入用户词典

```
In [7]: seg_list1 = jieba.cut("吴志祥是南京工业大学青年教师，他对那种二次元小魔仙是无感的，这怎么行？")

In [8]: print(' # '.join(seg_list1))

吴志祥#是#南京#工业#大学#青年教师#，#他#对#那种#二次元#小#魔仙#是#无感#的#，#这#怎么#行#？

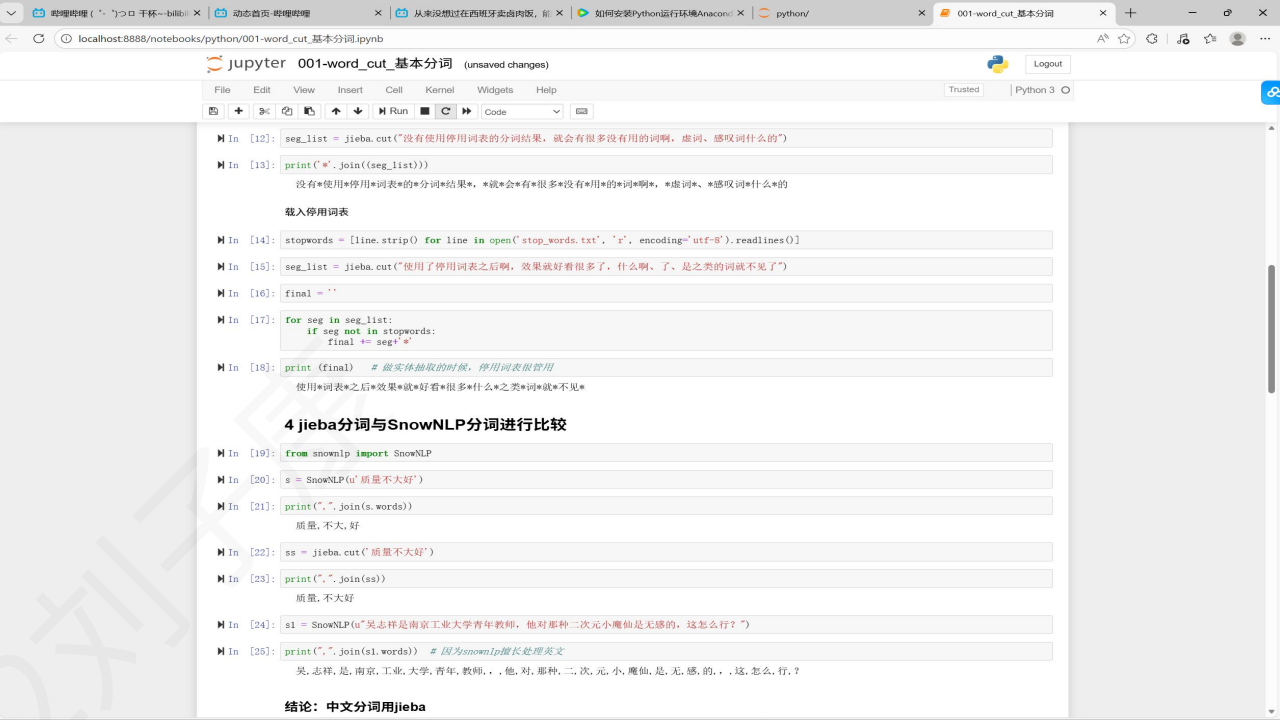
载入词典

In [9]: jieba.load_userdict('dict.txt')

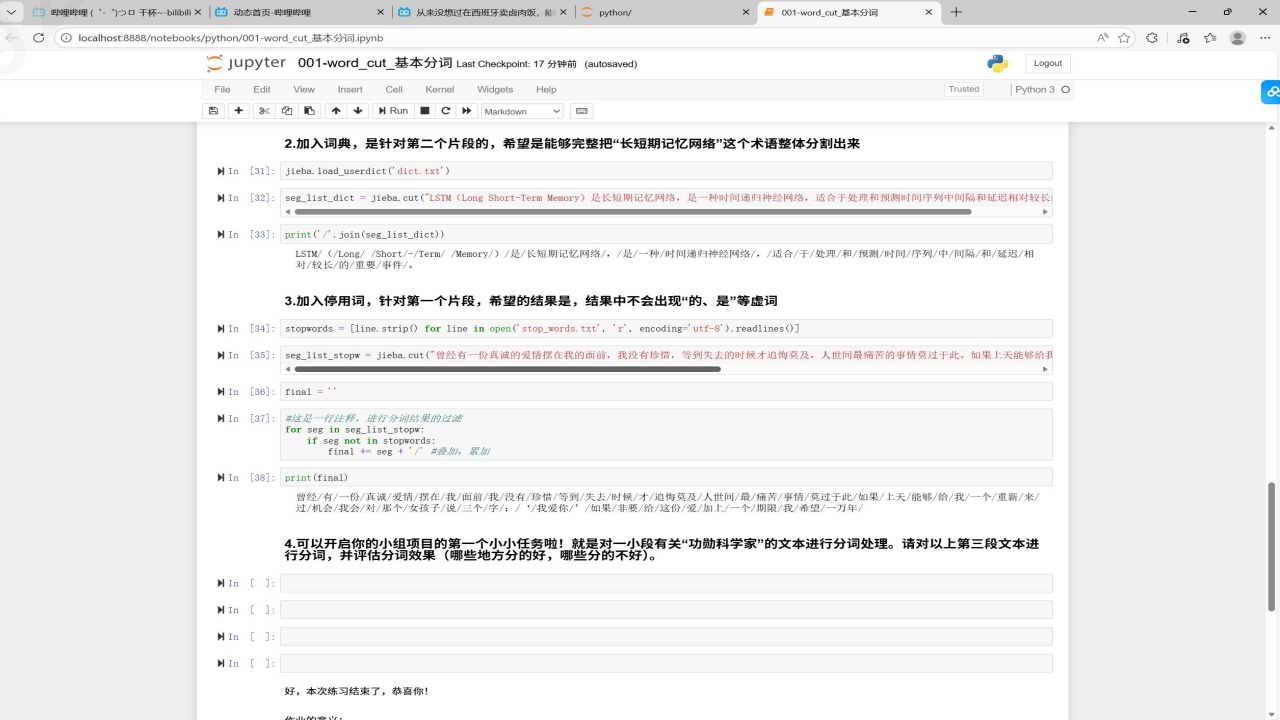
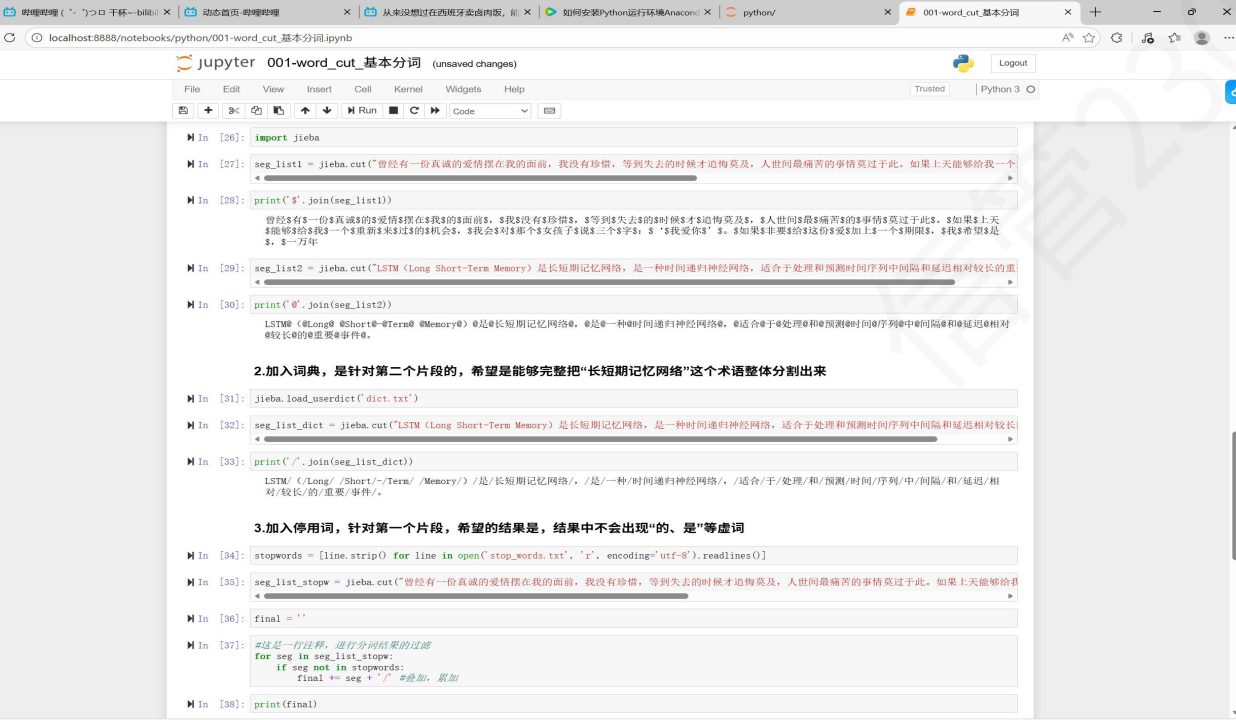
In [10]: seg_list = jieba.cut("吴志祥是南京工业最好大学青年教师，经济与管理学院的，他对那种二次元小魔仙是无感的，这怎么行？ python看上去还有点人性")

In [11]: print(' % '.join(seg_list))

吴志祥%是%南京工业最好大学%青年教师%，%经济%与%管理%学院%的%，%他%对%那种%二次元小魔仙%是%无感%的%，%这%怎么%行%？ %python%看上去%还%有点人性
```



结论：中文分词用jieba



002-word_cut_科学家文本 (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help

Not Trusted Python 3

功勋章科学家-黄旭华-传记文本分词

现在，可以开启你的小组项目的第一个小小任务啦！就是对一小段有关“功勋章科学家”的文本进行分词处理。

In [1]: # 简单分词

In [2]: import jieba

In [3]: seg_list_huang = jieba.cut('黄旭华, 1926年3月12日出生于广东省汕尾市, 原籍广东省揭阳市。1949年毕业于上海交通大学。历任北京海军核潜艇研究室副总工
«

In [4]: print(' '.join(seg_list_huang))

Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\HUAWEI\AppData\Local\Temp\jieba.cache
Loading model cost 1.142 seconds.
Prefix dict has been built successfully.
黄旭华/, /1926/年/3/月/12/日出/生于/广东省/汕尾市/, /原籍/广东省/揭阳市/, /1949/年/毕业/于/上海交通大学/, /历任/北京/海军/核潜艇/研究室/副/
总工程师/, /中/船重工/集团公司/核潜艇/总体/研究/设计所/研究员/, /名答/所长/, /1994/年/当选/为/中国工程院/院士/。

In [5]: # 加入用户词典

In [6]: jieba.load_userdict('dict.txt')

In [7]: seg_list_huang = jieba.cut('黄旭华, 1926年3月12日出生于广东省汕尾市, 原籍广东省揭阳市。1949年毕业于上海交通大学。历任北京海军核潜艇研究室副总工
«

In [8]: print(' '.join(seg_list_huang))

黄旭华/, /1926/年/3/月/12/日出/生于/广东省/汕尾市/, /原籍/广东省/揭阳市/, /1949/年/毕业/于/上海交通大学/, /历任/北京/海军/核潜艇/研究室/副/
总工程师/, /中/船重工/集团公司/核潜艇/总体/研究/设计所/研究员/, /名答/所长/, /1994/年/当选/为/中国工程院/院士/。

In [9]: # 加入词典之后, 哪些词汇被分出来了呢?

In [10]: # 使用停用词表

In [11]: stopwords = [line.strip() for line in open('stop_words.txt', 'r', encoding='utf-8').readlines()]

In [12]: stopwords = open('stop_words.txt', 'r', encoding='utf-8').read()
stopwords = stopwords.split('\n')

In [13]: stopwords

Out[13]: ['的', '了', '是', '啊', '、', '、', '、', '、', '停用']

002-word_cut_科学家文本 (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help

Not Trusted Python 3

In [8]: print(' '.join(seg_list_huang))

黄旭华/, /1926/年/3/月/12/日出/生于/广东省/汕尾市/, /原籍/广东省/揭阳市/, /1949/年/毕业/于/上海交通大学/, /历任/北京/海军/核潜艇/研究室/副/
总工程师/, /中/船重工/集团公司/核潜艇/总体/研究/设计所/研究员/, /名答/所长/, /1994/年/当选/为/中国工程院/院士/。

In [9]: # 加入词典之后, 哪些词汇被分出来了呢?

In [10]: # 使用停用词表

In [11]: # stopwords = [line.strip() for line in open('stop_words.txt', 'r', encoding='utf-8').readlines()]

In [12]: stopwords = open('stop_words.txt', 'r', encoding='utf-8').read()
stopwords = stopwords.split('\n')

In [13]: stopwords

Out[13]: ['的', '了', '是', '啊', '、', '、', '、', '、', '停用']

In [14]: seg_list_huang = jieba.cut('黄旭华, 1926年3月12日出生于广东省汕尾市, 原籍广东省揭阳市。1949年毕业于上海交通大学。历任北京海军核潜艇研究室副总工
«

In [15]: final = ''

In [16]: for seg in seg_list_huang:
if seg not in stopwords:
final+= seg+' '

In [17]: print(final)

黄旭华/1926/年/3/月/12/日出/生于/广东省/汕尾市/原籍/广东省/揭阳市/1949/年/毕业/于/上海交通大学/历任/北京/海军/核潜艇/研究室/副/总工程师/中船
重工集团公司/核潜艇/总体/研究/设计所/研究员/名答/所长/1994/年/当选/为/中国工程院院士/

作业的意义:

- 你可以处理比较复杂的文本啦
- 你开始尝试接触和理解, 一些具有文化内蕴的科技文献资源

In []:

In []:

In []:

In []:

In []:

The image displays two side-by-side screenshots of a Jupyter Notebook interface, showing the implementation of a word frequency analysis using Jieba.

Left Screenshot:

- Cell 4:** Imports `jieba` and `Counter` from `collections`.
- Cell 5:** Defines a text variable `text` containing a paragraph about enterprise management and digital transformation.
- Cell 6:** Executes `jieba.lcut(text)` to generate word tokens. The output shows the list of tokens, including '企业', '管理', '数字化', '智能化', '安全', etc.
- Cell 7:** Defines `target_words` as a list of words to analyze: `['数字化', '智能化', '安全']`.
- Cell 8:** Defines `word_counts` as a `Counter` object initialized with `words`.
- Cell 9:** Prints the word counts for the target words. The output shows the frequency of each word in the text.

Right Screenshot:

- Cell 10:** Prints the word counts for the target words. The output shows the frequency of each word in the text.
- Cell 11:** Prints the word counts for the target words. The output shows the frequency of each word in the text.

localhost:8888/notebooks/python/004_使用大语言模型提取科技文献中的实体.ipynb

jupyter 004_使用大语言模型提取科技文献中的实体 (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help

Not Trusted Python 3

In [2]:

```
import requests
import json

# 定义DeepSeek API的URL和headers
DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions"
API_KEY = "sk-78e64727760463394135e9246255cfa" #直接复制过来
```

In [3]:

```
# 准备prompt和论文文本
paper_text = """
随着肿瘤免疫微环境（Tumor Immune Microenvironment, TIME）研究的深入，
T细胞耗竭（T cell exhaustion）被认为是限制免疫治疗效果的关键机制之一。
本研究基于免疫编辑理论，提出了一种基于单细胞RNA测序（scRNA-seq）的T细胞状态动态识别方法。
具体而言，我们使用Seurat与Monocle3等生物信息学工具对50例非小细胞肺癌患者的肿瘤样本进行细胞亚群聚类和轨迹分析，
结合pseudotime推断T细胞从激活到耗竭的转化过程。此外，借助CellChat软件构建细胞间通讯网络，
进一步识别可能诱导T细胞耗竭的免疫抑制信号通路，如PD-1/PD-L1和TGF-β 路径。研究结果揭示了T细胞功能衰竭的关键节点，并为个体化免疫治疗提供了潜在靶点
"""

prompt = f"""
请从以下科技论文文本中提取包含理论、方法、工具的实体或专业术语，以json字典的格式输出：

{paper_text}
"""
```

In [4]:

```
# 准备请求数据
data = {
    "model": "deepseek-chat",
    "messages": [
        {"role": "user", "content": prompt}
    ],
    "temperature": 0.3
}

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {API_KEY}"
}

# 发送请求
response = requests.post(DEEPSEEK_API_URL, headers=headers, data=json.dumps(data))
```

In [5]:

```
# 处理响应
if response.status_code == 200:
    result = response.json()
    try:
        entities = result['choices'][0]['message']['content']
        print("提取到的实体和专业术语:")
        print(entities)
    except KeyError:
        print("无法解析API响应，原始响应:")
        print(response.text)
```

19:28

2025/12/29

localhost:8888/notebooks/python/004_使用大语言模型提取科技文献中的实体.ipynb

jupyter 004_使用大语言模型提取科技文献中的实体 (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Trusted Python 3

In [5]:

```
# 处理响应
if response.status_code == 200:
    result = response.json()
    try:
        entities = result['choices'][0]['message']['content']
        print("提取到的实体和专业术语:")
        print(entities)
    except KeyError:
        print("无法解析API响应，原始响应:")
        print(response.text)
else:
    print(f"请求失败，状态码: {response.status_code}")
    print(response.text)
```

提取到的实体和专业术语:

```
...json
{
  "理论": [
    "肿瘤免疫微环境",
    "T细胞耗竭",
    "免疫编辑理论"
  ],
  "方法": [
    "单细胞RNA测序",
    "细胞亚群聚类",
    "轨迹分析",
    "pseudotime推断",
    "细胞间通讯网络构建"
  ],
  "工具": [
    "Seurat",
    "Monocle3",
    "CellChat"
  ],
  "专业术语": [
    "TIME",
    "scRNA-seq",
    "非小细胞肺癌",
    "PD-1/PD-L1",
    "TGF-β 路径",
    "免疫抑制信号通路",
    "个体化免疫治疗"
  ]
}
```

提问:

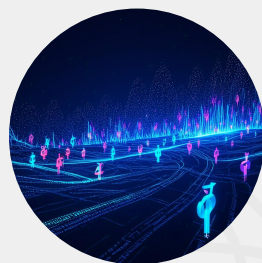
- 1, 使用deepseek开展工作的感觉如何?
- 2, 你觉得大语言模型的活干的怎么样?

基于关键词的学术文本聚类集成研究



研究背景

文本聚类是无监督高效的文本类别划分方法，基于关键词的聚类为主流方式；现有学术文本聚类多依赖单一方法，聚类集成是提升性能的新路径（研究发表于《情报学报》CSSCI）。



核心实验设计

对比非集成聚类与聚类集成算法性能差异；同步分析TextRank等不同关键词抽取方法、关键词个数变化对聚类结果的影响。



关键研究结论

聚类集成算法显著提升学术文本聚类性能；TextRank为较优关键词抽取方法；关键词个数增加可同步优化文本类别划分效果。

第二讲

The image displays two side-by-side screenshots of a Jupyter Notebook interface, showing the process of text preprocessing in Chinese.

Left Screenshot:

- The notebook is titled "jupyter tf1_full_text_huangxuhua (autosaved)".
- The code cells show the following steps:
 - Importing `jieba`.
 - Opening and reading a text file: `article = open('科学家博物馆-黄旭华传记序言.txt', 'r', encoding='utf-8').read()`. A comment indicates that the file contains a hint to convert the encoding from `ANSI` to `utf-8`.
 - Deleting punctuation and symbols: `delete = ['.', '!', '?', '的', '“', '”', '‘', '’', '《', '》', '《', '》', '‘', '’', '']`. A comment says: "手动设计一些停用词和符号".
 - Adding a word to the dictionary: `jieba.add_word('国立交通大学')`. A comment says: "加入字典中没有的新词".
 - Converting the text to a list of words: `words = list(jieba.cut(article))`. A comment says: "结巴分词出来的词汇".
 - Printing the words: `words`.
- The output shows a list of words, including: `['在', '核潜艇', '领域', '我国', '已', '形成', '一套', '完整', '的', '研究', '设计', '试验', '制造', '测试']`.
- A section titled "字典" (Dictionary) shows the creation of an empty dictionary: `articleDict = {}`. A comment says: "这是一个字典，准备词-词频的保存".
- A section titled "集合 (数据结构的一种)" (Set (a type of data structure)) shows the creation of a set: `articleSet = set(words)-delete`.
- The output shows the resulting set: `articleSet` containing words like "不是", "不立", "与", "专业", "专业技能", "专家".

Right Screenshot:

- The notebook is titled "jupyter tf1_full_text_huangxuhua (autosaved)".
- The code cells show the following steps:
 - Iterating over the words and counting their frequency: `for w in articleSet: if len(w)>1: articleDict[w] = words.count(w)`. A comment says: "字典的key-value配对".
 - Printing the dictionary: `articleDict`.
- The output shows the resulting dictionary: `Out[15]: {'揭示': 1, '加入': 2, '承担': 2, '主旨': 1, '当时': 1, '性能': 2, '经验': 1, '仿制': 1, '齐心协力': 1, '组织': 2, '命名': 1, '七两': 1, '包括': 3, '清晰': 1, '动荡': 1, '还有': 1, '树感': 1, '品德': 2, '成立': 1}`.
- A section titled "字典" (Dictionary) shows the sorting of the dictionary: `articlelist = sorted(articleDict.items(), key = lambda x:x[1], reverse = True)`. A comment says: "对字典中的词排序".
- The output shows the sorted list: `Out[16]: [('黄旭华', 53), ('核潜艇', 32), ('采梨', 29), ('学术', 22), ('资料', 21), ('工作', 17), ('成长', 15), ('小组', 14), ('院士', 13), ('进行', 13), ('专业', 13), ('技术', 12), ('研制', 12), ('我国', 12), ('工程', 11), ('访谈', 10), ('第一代', 8), ('介绍', 8), ('主要', 8)]`.

python/

tf1_full_text_sanguo

tf1_full_text_huangxuhua

+

python/tf1_full_text_sanguo.ipynb

jupyter tf1_full_text_sanguo (autosaved)

Logout

File Edit View Insert Cell Kernel Widgets Help

Trusted Python 3 C

+

%

📄

📄

⬆️

⬆️

Run

🔄

Code

📄

In [1]:

!pip install jieba

Requirement already satisfied: jieba in d:\anaconda\lib\site-packages (0.42.1)

In [2]:

import jieba

In [3]:

article = open('sanguo_10.txt', 'r', encoding = 'utf-8').read() # ansi/打开并读取三国前10回 #出现是吗提示, 就把ANSI改成utf-8

In [4]:

dele = ['。', '!', '?', '的', '“', '”', '(', ')', '‘', '’', ':', ';', ','] # 手动设计一些停用词和符号

In [5]:

jieba.add_word('羅貫中') # 加入字典中没有的新词

Building prefix dict from the default dictionary ...

Loading model from cache C:\Users\HUAWEI\AppData\Local\Temp\jieba.cache

Loading model cost 2.157 seconds.

Prefix dict has been built successfully.

In [6]:

words = list(jieba.cut(article)) # 结巴分词出来的词汇

In [7]:

words

['秦滅',
'之',
'後',
'.',
'楚',
'.',
'漢分乎',
'.',
'又',
'並入',
'於',
'漢',
'.',
'漢朝',
'自',
'高祖',
'.',
'斬',
'白蛇',
'.'

字典

In [8]:

articleDict = {} # 这是一个字典, 准备词-词频的保存

集合 (数据结构的一种)

DeepSeek 开发平台

python/

tf1_full_text_sanguo

tf1_full_text_huangxuhua

+

localhost:8888/notebooks/python/tf1_full_text_sanguo.ipynb

jupyter tf1_full_text_sanguo (autosaved)

Logout

File Edit View Insert Cell Kernel Widgets Help

Trusted Python 3 O

+

%

📄

📄

⬆️

⬆️

Run

🔄

Code

📄

In [14]:

articledict = sorted(articleDict.items(), key = lambda x:x[1], reverse = True) # 对词典中的词排序

In [15]:

输出词频的前10个

for i in range(100):

print(articledict[i])

['塞罕', 97]
['吕布', 60]
['曹操', 58]
['袁绍', 57]
['天下', 53]
['玄德', 48]
['劉禪', 37]
['太子', 36]
['朝廷', 32]
['不可', 31]
['孫堅', 31]
['沈丘', 26]
['南陽', 25]
['引兵', 25]
['李福', 25]
['天子', 24]
['左右', 23]
['玄德曰', 22]
['太叔', 22]
['今日', 21]
['大將', 21]
['校尉', 21]
['軍士', 21]
['王允', 20]
['太后', 20]
['何進', 20]
['二人', 19]
['不能', 18]
['劉備', 18]
['司徒', 18]
['公孫瓚', 18]
['朝廷', 18]
['太常', 17]
['軍師', 17]
['何人', 17]
['百姓', 17]
['直隸', 17]
['如此', 17]
['離臣', 17]
['百姓', 16]
['李福', 16]
['劉禪', 16]
['大將', 15]
['劉備', 15]
['如何', 15]
['何處', 15]
['何人', 15]

python/

tf1_full_text_sanguo

tf1_full_text_huangxuhua

+

python/tf1_full_text_sanguo.ipynb

jupyter tf1_full_text_sanguo (autosaved)

Logout

File Edit View Insert Cell Kernel Widgets Help

Trusted Python 3 O

+

%

📄

📄

⬆️

⬆️

Run

🔄

Code

📄

In [9]:

articleSet = set(words)-dele

In [10]:

articleSet

['近聞十常',
'勢大敗',
'張梁',
'之助乎',
'溫結連',
'追殺',
'可克日',
'相遇',
'名宮',
'截',
'無疑',
'留人',
'一人出',
'自退',
'將崩今後土',
'既而席',
'猛虎',
'手下',
'溫頭',
'自去']

In [11]:

因为采用的是循环的方法, 把所有的词都循环一遍, 长度大于1, 所以速度很慢!

In [12]:

for w in articleSet:

if len(w)>1:

articledict[w] = words.count(w) # 字典的key-value配对

In [13]:

articledict

['一人出': 1,
'自退': 1,
'將崩今後土': 1,
'既而席': 1,
'猛虎': 1,
'手下': 4,
'溫頭': 1,
'自去': 1,
'宜卓入': 1,
'不題': 1,
'獲得些': 1,
'憤怒': 1,
'貢進': 1,
'四月': 4,
'武揚威': 1,
'袁紹': 57,
'相國': 1,
'紹軍並': 1,
'進恐': 1,
'收發糧'...

python/

tf2_the_three_kingoms

+

s/python/tf2_the_three_kingoms.ipynb

jupyter tf2_the_three_kingoms (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help

Not Trusted Python 3

+

⌕

↶

↷

⏮

⏭

⏪

⏩

Run

⏹

Code

⌵

¶ In [1]:

文件的打开与关闭

¶ In [2]:

f_name = open('name.txt', encoding = 'GB18030') #使用mac的小伙伴, 需要耐心测试下编码吗GB18030

¶ In [3]:

f_name = open('name.txt')

¶ In [4]:

data_name = f_name.read()

¶ In [5]:

data_name[:70]

Out[5]:

'諸葛亮|關羽|劉備|曹操|孫權|關羽|張飛|呂布|周瑜|趙雲|龐統|司馬懿|黃忠|馬超'

¶ In [6]:

print(data_name[:30])

諸葛亮|關羽|劉備|曹操|孫權|關羽|張飛|呂布|周瑜|趙雲|龐統|司馬懿|黃忠|馬超

¶ In [7]:

f_name.close()

¶ In [8]:

將文本轉化為列表

¶ In [9]:

names = data_name.split('|') # split-分names就是列表

¶ In [10]:

print(names)

['諸葛亮', '關羽', '劉備', '曹操', '孫權', '關羽', '張飛', '呂布', '周瑜', '趙雲', '龐統', '司馬懿', '黃忠', '馬超']

¶ In [11]:

names

Out[11]:

['諸葛亮',
'關羽',
'劉備',
'曹操',
'孫權',
'關羽',
'張飛',
'呂布',
'周瑜',
'趙雲',
'龐統',
'司馬懿',
'黃忠',
'馬超']

¶ In [12]:

学习一种新的数据结构: 字典

¶ In [13]:

name_dict = {}

¶ In [14]:

f_txt = open('sanguo.txt', encoding = 'GB18030')

¶ In [15]:

data_txt = f_txt.read()

¶ In [16]:

f_txt.close()

¶ In [17]:

print(data_txt[:100])

《三國演義》(全)

(明)羅貫中著

第一回
宴桃園豪傑三結義斬黃巾英雄首立功
話說天下大勢，分久必合，合久必分。周末七國分爭，並
入于秦，及秦滅之後，楚、漢分爭，又並入於漢；漢朝自高祖
斬白

¶ In [18]:

用count函数统计文本中的词汇

你没有发现下面的代码很神奇吗? data_txt.count一下, 就可以统计词! 实际上是字符串匹配的过程!

¶ In [19]:

for name in names:
 name_dict[name]=data_txt.count(name)

¶ In [20]:

name_dict

Out[20]:

{'諸葛亮': 149,
'關羽': 8,
'劉備': 291,
'曹操': 907,
'孫權': 315,
'張飛': 347,
'呂布': 332,
'周瑜': 235,
'趙雲': 301,
'龐統': 80,
'司馬懿': 272,
'黃忠': 179,
'馬超': 212}

¶ In [21]:

定义 函数 函数

¶ In [22]:

def make_chinese_plot_ready():
 from matplotlib import rcParams
 rcParams['font.family'] = 'Heiti TC' # mac笔记本电脑直接使用繁体字体
 #rcParams['font.sans-serif'] = ['FangSong'] # 或者直接使用使用电脑有的字体 FangSong
 rcParams['axes.unicode_minus'] = False

¶ In [23]:

¶ In [24]:

¶ In [25]:

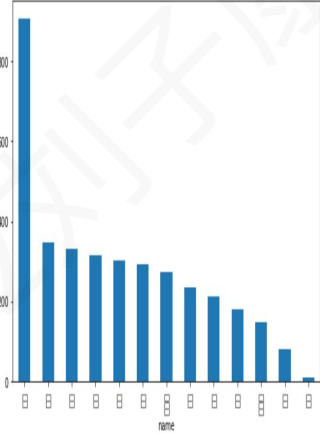
¶ In [26]:

%matplotlib inline

¶ In [27]:

draw_dict(name_dict)

D:\Anaconda\lib\site-packages\matplotlib\font_manager.py:1331: UserWarning: findfont: Font family ["Heiti TC"] not found. Falling back to De
jaVu Sans
(prop.get_family(), self.defaultFamily[fontext])



¶ In []:

2 统计武器

统计武器的步骤

- 打开武器文本
- 对文本进行处理, 包括: 去掉换行符, 去掉空格, 变成列表
- 打开文本
- 统计武器 (采用文本与武器的字符进行匹配的方式, 统计的. 没有分词)
- 用字典存储结果: 武器-频次
- 展示

¶ In [28]:

f_weapon = open('weapon.txt', encoding = 'utf-8')

¶ In [29]:

data_weapon = f_weapon.read()

¶ In [30]:

print(data_weapon[:100])

python/tf2_the_three_kingoms.ipynb

jupyter tf2_the_three_kingoms (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Trusted Python 3 O

+

⌕

↶

↷

⏮

⏭

⏪

⏩

Run

⏹

Code

⌵

¶ In [28]:

f_weapon = open('weapon.txt', encoding = 'utf-8')

¶ In [29]:

data_weapon = f_weapon.read()

¶ In [30]:

print(data_weapon[:100])

青龍偃月刀

丈八點鋼矛

鐵脊蛇矛

涯角槍

諸葛槍

方天畫戟

長柄鐵錘

鐵索藤骨索

大斧

蘸金斧

三尖刀

截頭大刀

馬岱寶刀

古錠刀

衝鋒槊

丈八長標

王雙大刀

呂俊刀

¶ In [31]:

weapons_origin = data_weapon.split('\n')

¶ In [32]:

print(weapons_origin[:40])

['青龍偃月刀', '', '丈八點鋼矛', '', '鐵脊蛇矛', '', '涯角槍', '', '諸葛槍', '', '方天畫戟', '', '長柄鐵錘', '', '鐵索藤骨索', '', '大斧',
'', '蘸金斧', '', '三尖刀', '', '截頭大刀', '', '馬岱寶刀', '', '古錠刀', '', '衝鋒槊', '', '丈八長標', '', '王雙大刀', '', '呂俊刀', '',
'龍泉劍', '', '倚天劍', '']

¶ In [33]:

weapons = []

鐵鞭：1，
鋼鞭：2，
四楞鐵筒：1，
雙鐵戟：2，

Out[38]:

青龍擺月刀:	2.
丈人貼鬚牙:	4.
鑽背蛇矛:	1.
涯角槓:	0.
諸葛扇:	0.
方天畫戟:	2.
長柄鐵鎚:	0.
鐵索多骨索:	0.
大客:	14.
雁金斧:	1.
三尖刀:	1.
截頭大刀:	1.
馬岱寶刀:	0.
古錠刀:	1.
衡綱鞭:	0.
丈人長標:	1.
王雙刀:	0.
呂虔刀:	0.
龍泉劍:	0.
倚天劍:	1.
青虹:	0.
七寶刀:	1.
雙股劍:	3.
松紋幽寶劍:	0.
孟德劍:	0.
思召劍:	0.
飛景三劍:	0.
文士劍:	0.
蜀八劍:	0.
鎮山劍:	0.
吳六劍:	0.
皇帝吳王劍:	0.
日月刀:	1.
百辟寶刀:	0.
龍鱗刀:	0.
百辟七首二:	0.
鐵鞭:	2.
鋼鞭:	2.
四楞鐵鞭:	1.
雙鐵戟:	2.

► In []:

python/

tf2_the_huangxuhua

+

python/tf2_the_huangxuhua.ipynb

jupyter tf2_the_huangxuhua (unsaved changes)

Logout

File Edit View Insert Cell Kernel Widgets Help

Not Trusted Python 3

+

↺

↻

↵

↴

↱

↲

↳

Run

⏏

↺

↻

↵

↴

↲

Code

⌵

⌵

⌵ In [1]:

文件的打开与关闭

⌵ In [2]:

#f_name = open('name.txt', encoding = 'GB18030') #使用mac的小伙伴, 需要耐心调试下编码GB18030

⌵ In [3]:

#f_name = open('name.txt')

⌵ In [4]:

#data_name = f_name.read()

⌵ In [5]:

#data_name[:70]

⌵ In [6]:

#print(data_name[:50])

⌵ In [7]:

#f_name.close()

⌵ In [8]:

将文本转化为列表

⌵ In [9]:

#names = data_name.split('/') # split一下names就是列表

⌵ In [10]:

以上是课程演示里面的方法, 特定的需要统计的词汇放置在txt文本中
我们需要统计的词汇很少, 就直接定义需要统计的词汇

⌵ In [11]:

terms = ['黄旭华', '核潜艇', '国立交通大学']

⌵ In [12]:

print(terms)

Out[13]:

['黄旭华', '核潜艇', '国立交通大学']

⌵ In [13]:

terms

⌵ In [14]:

学习一种新的数据结构, 字典

⌵ In [15]:

terms_dict = {}

⌵ In [16]:

f_txt = open('科学家博物馆-黄旭华传记序言.txt', encoding = 'utf-8')

⌵ In [17]:

data_txt = f_txt.read()

⌵ In [18]:

f_txt.close()

⌵ In [19]:

print(data_txt[:1000])

在核潜艇领域, 我国已形成一套完整的研究、设计、试验、制造、测试的核潜艇产业体系, 而且装备了一支具有极高战略威慑力的、成梯次配备的、已近实现战备巡逻的核潜艇部队。回顾我国核潜艇的发展历程, 人们自然会想起以黄旭华为代表的五位两院院士及无数第一代核潜艇研制人员的皓首穷经、筚路蓝缕、无私奉献, 正是他们所铸就的国之重器使我国彻底摆脱了超级大国的核讹诈, 更使我们在民族复兴的道路上迈出了坚实的一步。

python/

tf2_the_huangxuhua

+

python/tf2_the_huangxuhua.ipynb

jupyter tf2_the_huangxuhua (unsaved changes)

Logout

File Edit View Insert Cell Kernel Widgets Help

Not Trusted Python 3

+

↺

↻

↵

↴

↱

↲

↳

Run

⏏

↺

↻

↵

↴

↲

Code

⌵

⌵

⌵ In [23]:

定义 画图 函数

⌵ In [24]:

def make_chinese_plot_ready():
 from matplotlib import rcParams
 rcParams['font.family'] = 'Heiti TC' # mac笔记本电脑直接替换字体
 #rcParams['font.sans-serif'] = ['FangSong'] # 或者直接使用电脑有的字体 FangSong
 rcParams['axes.unicode_minus'] = False

⌵ In [25]:

定义 画图 函数

⌵ In [26]:

def draw_dict(mydict, figsize=(8, 5)):
 import pandas as pd
 import matplotlib.pyplot as plt
 make_chinese_plot_ready()
 df = pd.DataFrame(list(mydict.items()), columns=['name', 'times'])
 df.set_index('name')['times'].sort_values(ascending=False).plot(kind='bar', figsize=figsize) # 做好排序
 plt.tight_layout()

⌵ In [27]:

%pylab inline

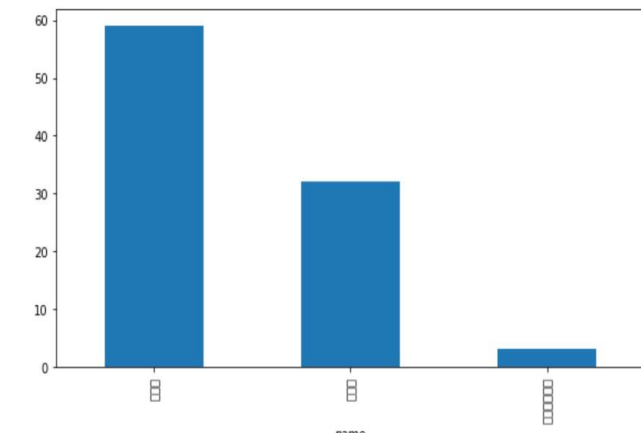
⌵ In [28]:

%matplotlib inline

⌵ In [29]:

draw_dict(terms_dict)

D:\Anaconda\lib\site-packages\matplotlib\font_manager.py:1331: UserWarning: findfont: Font family ['Heiti TC'] not found. Falling back to DejaVu Sans
(prop.get_family(), self.defaultFamily[fonttext]))



name	times
黄旭华	59
核潜艇	32
国立交通大学	3

阅读总结



词频统计与文本精准解读

张一兵研究以词频统计为核心方法，2012年基于德文原文统计马克思哲学文本核心概念频次，为“两次转变论”提供实证支持；2015年拓展至福柯法文文本，通过关键词词频分析揭示思想构境演变，提出“阅读复权”理念，强调立足母语原文避免翻译偏差的重要性。



替代计量与跨圈影响力评估

Wang等2018年借助谷歌数字化图书与学术文献，追踪顶尖物理学家科学声誉，发现伟大科学家影响力跨世纪延续且存在“群体内偏好”现象，验证谷歌图书与Ngram Viewer作为替代计量工具的有效性，为评估学者跨学术圈影响力提供新视角。



算法行为分析与工具选择参考

章成志等2018年聚焦自然语言处理领域，从使用频次、位置、年代和动机四维度分析十大经典数据挖掘算法使用行为，结果显示SVM算法影响力最高，CART与Apriori较低，为研究者算法选择提供数据化参考依据。



量化方法的学术研究价值拓展

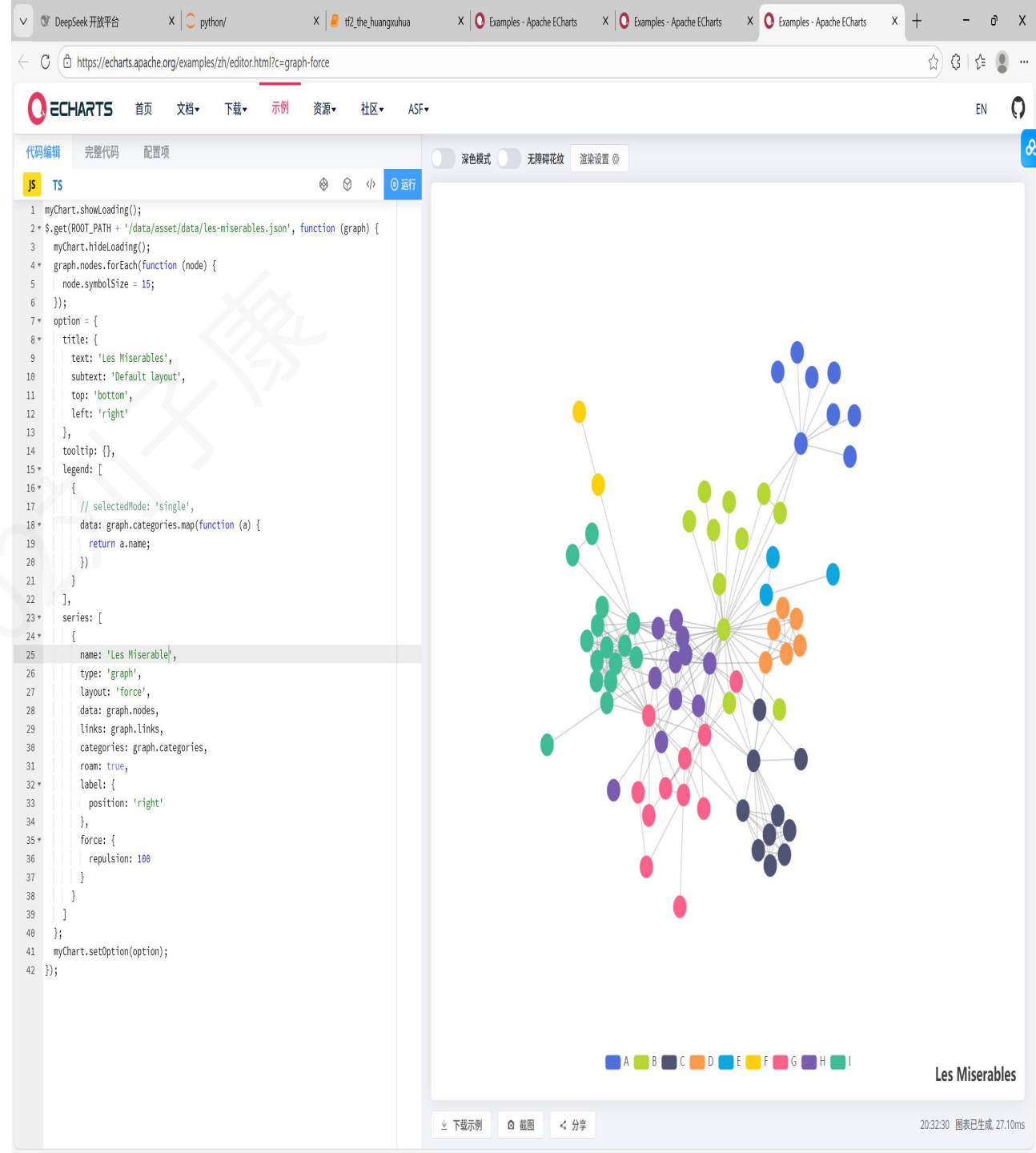
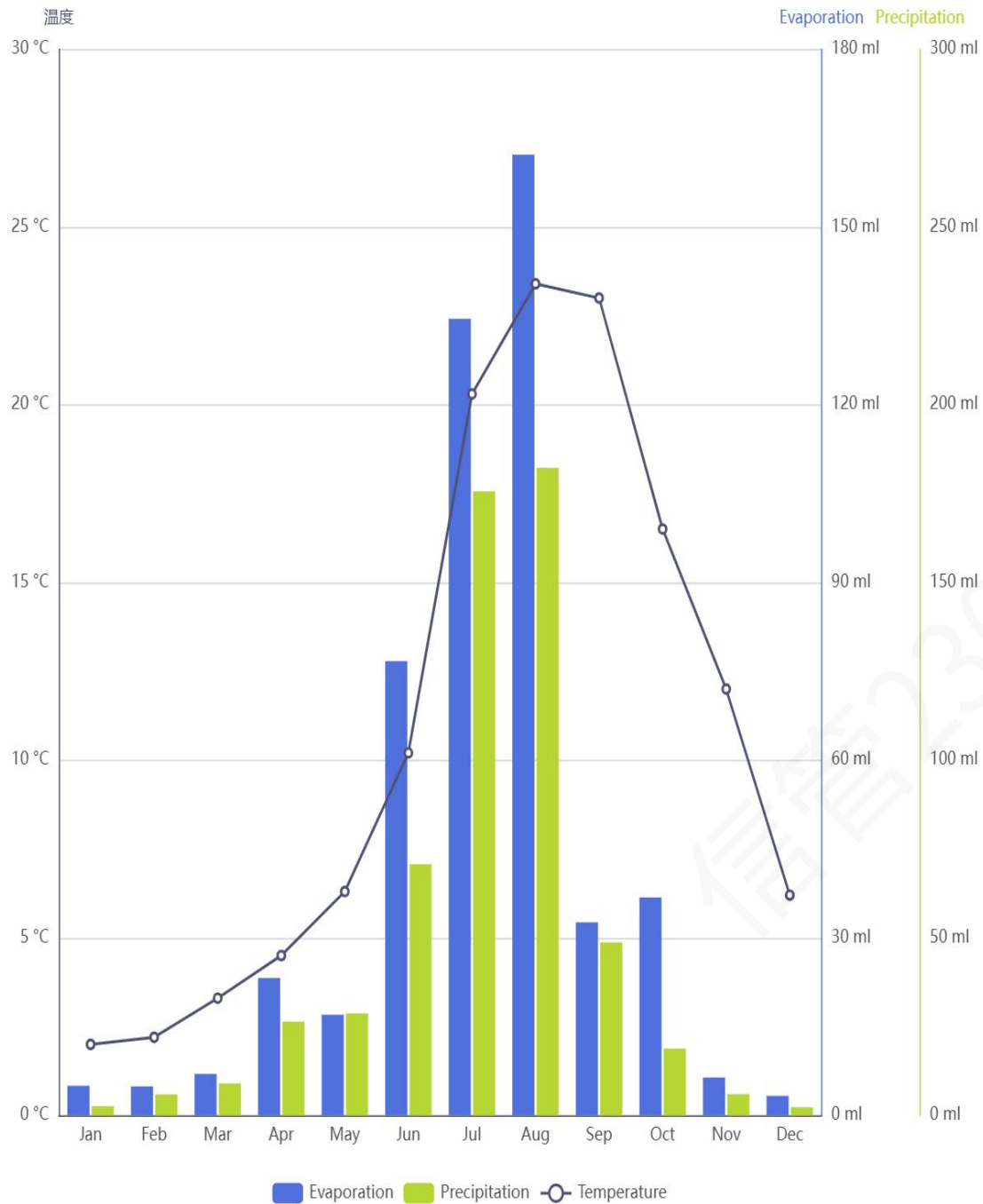
四篇文献共同体现量化分析的学术价值：通过词频统计、数据挖掘等方法，为文本深度解读、算法工具评估和学者声誉研究提供客观实证支撑，突破传统定性研究局限，系统性拓展相关领域研究视角与方法论边界。

第三讲

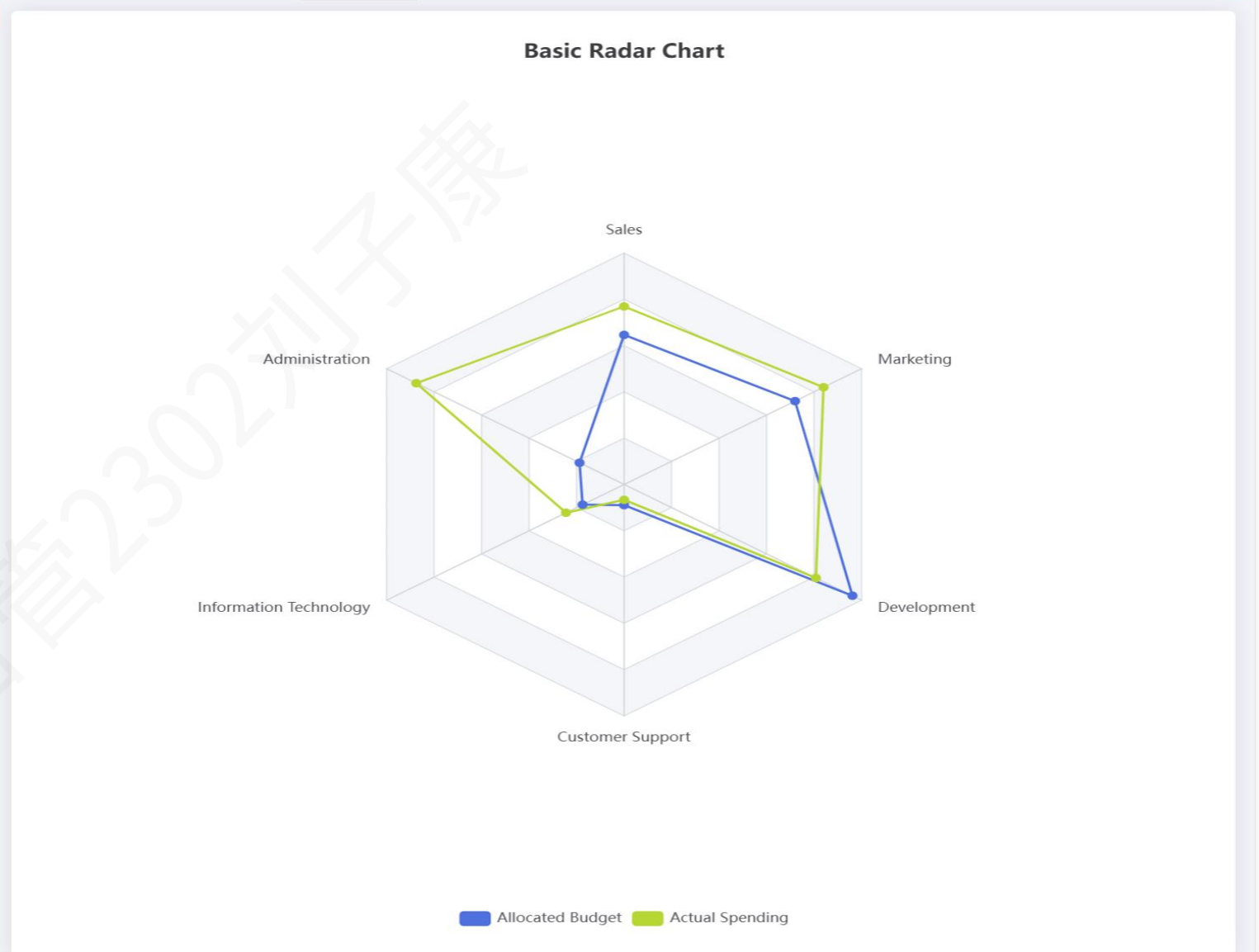


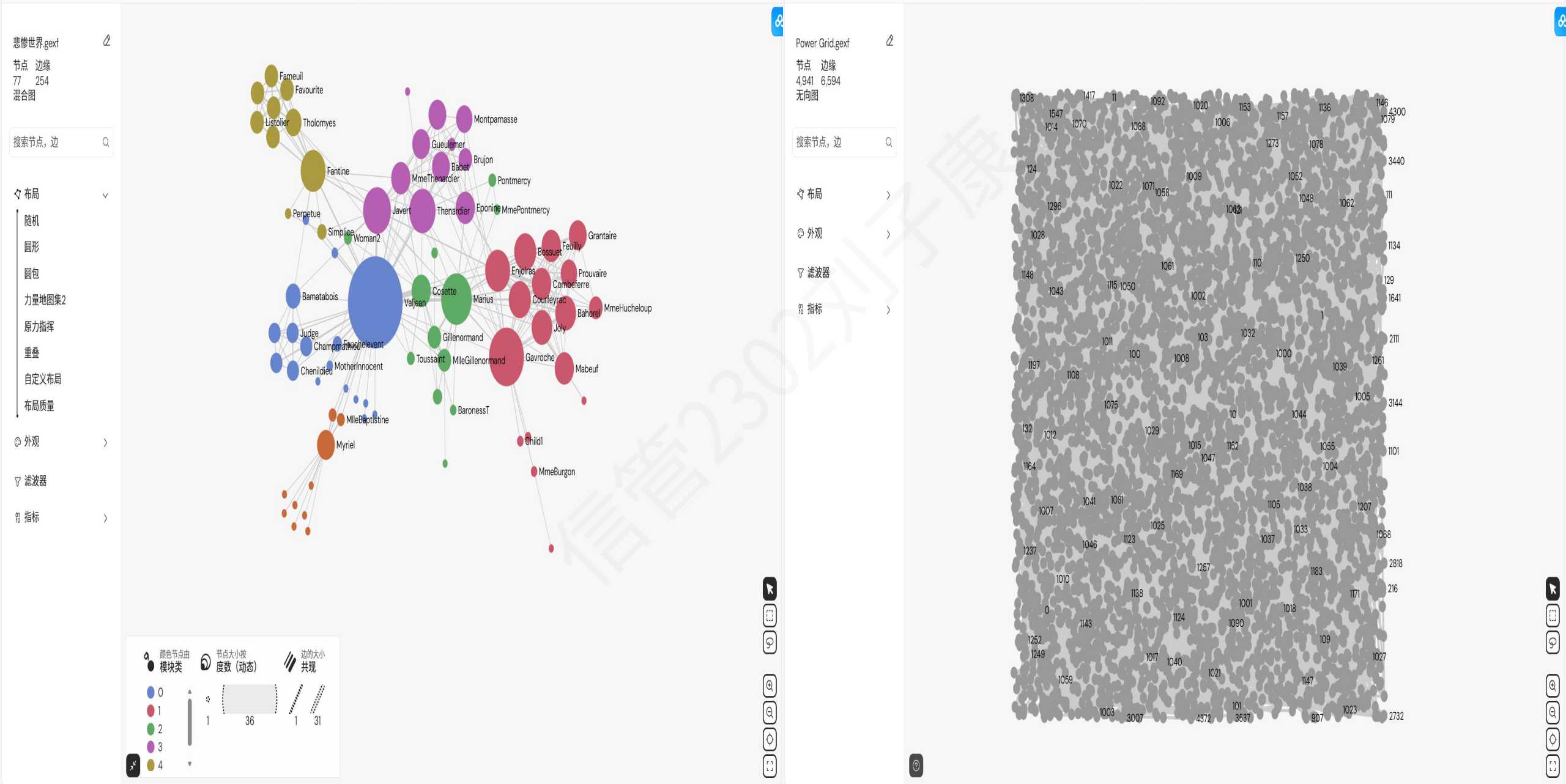
这是一幅以“玫瑰”为轮廓的词云图，核心呈现的是与阿加莎小说相关的高频元素，可从三方面解读：空间场景：“客厅”“花园”“书房”“露台”等室内外场所高频出现，构成了故事的核心发生环境；地点标识：“芬利庄园”“利物浦”“波洛住所”等专有地点，明确了叙事的地理与空间范围；关键关联元素：“三只野猪”“金斯艾伯特”等独特名称，或是故事中的关键符号、人物关联地点，强化了叙事的专属标识性。整体词云通过高频词汇的聚合，直观呈现了该叙事中“场景+地点+专属符号”的核心要素，体现了其空间与情节的聚焦性。

词云图，也叫文字云或标签云，是一种将文本数据进行可视化呈现的图形。它的核心逻辑是以词语出现的频率为核心依据，通过调整词语的字号大小、颜色、字体等视觉元素，直观地展示文本中关键词的重要程度——在文本中出现次数越多的词语，字号通常越大、位置越醒目；出现频率较低的词语，则字号相对较小，排在次要位置。词云图的优势在于能快速提炼文本核心信息，将枯燥的文字数据转化为简洁易懂的可视化图形，帮助观察者在短时间内捕捉文本的重点内容。比如在国产动漫受众调研中，将观众的评论文本生成词云图后，“剧情紧凑”“画面精良”“人物立体”这类高频评价会以更大的字号呈现，直观反映出观众对作品的核心反馈。同时，词云图还可以通过自定义颜色搭配和排版风格，适配不同的展示场景，兼具实用性与观赏性。




```
1 option = {
2   title: {
3     text: 'Basic Radar Chart'
4   },
5   legend: {
6     data: ['Allocated Budget', 'Actual Spending']
7   },
8   radar: {
9     // shape: 'circle',
10    indicator: [
11      { name: 'Sales', max: 6500 },
12      { name: 'Administration', max: 16000 },
13      { name: 'Information Technology', max: 114514 },
14      { name: 'Customer Support', max: 391981 },
15      { name: 'Development', max: 52000 },
16      { name: 'Marketing', max: 25000 }
17    ]
18  },
19  series: [
20    {
21      name: 'Budget vs spending',
22      type: 'radar',
23      data: [
24        {
25          value: [4200, 3000, 20000, 35000, 50000, 18000],
26          name: 'Allocated Budget'
27        },
28        {
29          value: [5000, 14000, 28000, 26000, 42000, 21000],
30          name: 'Actual Spending'
31        }
32      ]
33    }
34  ]
35 };
```





第四讲

在线演示

中文分词

词性标注

命名实体识别

依存句法分析

成分句法分析

语义依存分析

语义角色标注

抽象意义表示

指代消解

语义文本相似度

文本风格转换

关键词短语提取

抽取式自动摘要

生成式自动摘要

文本纠错

文本分类

情感分析

HanLP / 演示 / 情感分析

情感分析

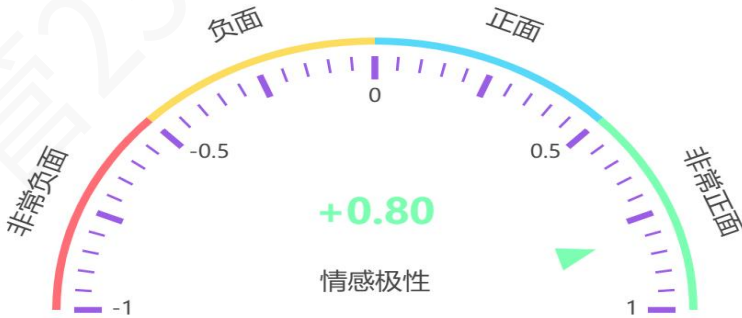
请输入一段中文文本：

同学们，下周一（12月8号），下午2：30-5：00，文科楼447教室。我邀请南京农业大学张卫老师过来讲座。分享下他近期有关物质文化遗产叙事文本语义组织方面的工作。欢迎有兴趣的同学跟我报名，限额3位，先到先得。

105/1000

情感分析

情感极性



情感极性

简介

情感分析（Sentiment Analysis）任务的目标是判断一段文本的情感极性。情感极性为 $[-1, 1]$ 之间的数值，数值的正负代表正负面情绪，数值的绝对值代表情感的强烈程度。

此页内容

简介

调用方法

创建客户端

情感分析

本地调用

多语种支持


```
In [1]: import pandas as pd # pandas用来处理表格数据的工具包
```

```
In [3]: df = pd.read_excel("restaurant-comments.xlsx")
```

```
In [4]: df.head()
```

```
Out[4]:
```

	comments	date
0	这辈子最爱吃的火锅，一星期必吃一次啊！最近才知道他家还有免费鸡蛋羹.....炒鸡好吃炒鸡嫩...	2017-05-14 16:00:00
1	第N次来了，还是喜欢?..... 从还没上A餐厅的楼梯开始，服务员已经在那迎宾了，然...	2017-05-10 16:00:00
2	大烧过生日，姐姐定的这家A餐厅的包间，服务真的是没得说，A餐厅的服务也是让我由衷的欣赏，很久...	2017-04-20 16:00:00
3	A餐厅的服务哪家店都一样，体贴入微。这家店是我吃过的排队最短的一家，当然也介于工作日且比较晚...	2017-04-25 16:00:00
4	因为下午要去天津站接人，然后我倩前几天就说想吃A餐厅，然后正好这有，就来这吃了。 来的...	2017-05-21 16:00:00

```
# 注意这里的时间列。如果你的Excel文件里的时间格式跟此处一样，包含了日期和时间，那么Pandas会非常智能地帮你把它识别为时间格式，接着往下做就可以了。

反之，如果你获取到的时间只精确到日期，例如"2017-04-20"这样，那么Pandas只会把它当做字符串，后面的时间序列分析无法使用字符串数据。解决办法是在这里加入以下两行代码：

from dateutil import parser
df["date"] = df.date.apply(parser.parse)

这样，你就获得了正确的时间数据了
```

```
In [6]: text = df.comments.iloc[0] # iloc是索引（用来定位到指定的位置），这里就是第一条评论文本
```

```
In [7]: text
```

```
Out[7]: '这辈子最爱吃的火锅，一星期必吃一次啊！最近才知道他家还有免费鸡蛋羹.....炒鸡好吃炒鸡嫩啊！！新出的红皮土豆也好好吃，还有炸酥肉，秒杀任何火锅店啊！服务员太可爱，告诉我们半份豆花是4块儿，一份豆花是6块儿，点两个半份比较合适，太实在了哈哈哈，每次妈妈说开心果好吃服务员都给我们打包带走??希望A餐厅早日出咖喱锅，期待ing.....'
```

```
In [8]: from snownlp import SnowNLP
```

```
In [9]: s = SnowNLP(text)
```

```
In [11]: s.sentiments
```

```
Out[11]: 0.4244401030222834
```

情感分析数值可以正确计算。在此基础上，我们需要定义函数，以便批量处理所有的评论信息。

```
In [12]: def get_sentiment_cn(text):
          s = SnowNLP(text)
          return s.sentiments # snownlp是通用的，就没那么准确
```

```
In [16]: df.sentiments.median()
```

```
Out[16]: 0.9270364310550024
```

我们发现了有趣的现象——中位数值不仅比平均值高，而且几乎接近1（完全正面）。这就意味着，大部分的评价一边倒表示非常满意。但是存在着少部分异常点，显著拉低了平均值。下面我们情感的时间序列可视化功能，直观查看这些异常点出现在什么时间，以及它们的数值究竟有多低。

我们需要使用ggplot绘图工具包。这个工具包原本只在R语言中提供，让其他数据分析工具的用户羡慕得流口水。幸好，后来它很快被移植到了Python平台。

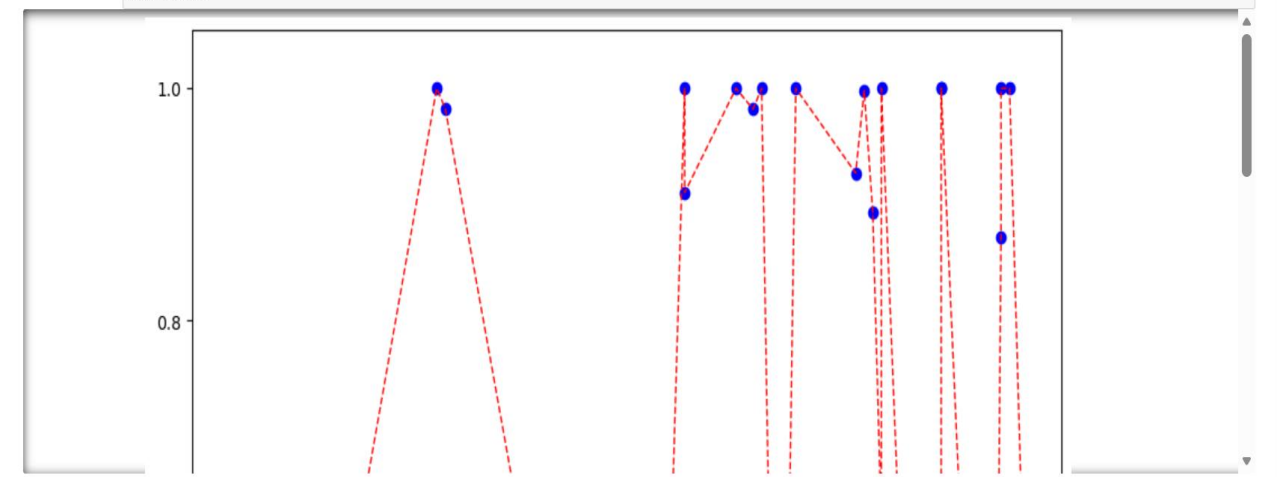
我们从ggplot中引入绘图函数，并且让Jupyter Notebook可以直接显示图像。

```
In [17]: %pylab inline
```

```
%pylab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib
```

```
/Users/wuzhixiang/anaconda3/lib/python3.11/site-packages/IPython/core/magics/pylab.py:162: UserWarning: pylab import has clobbered these variables: ['text']
`%matplotlib` prevents importing * from pylab and numpy
warn("pylab import has clobbered these variables: %s" % clobbered +
```

```
In [18]: import matplotlib.pyplot as plt
          #plt.scatter(df["date"],df["sentiments"],color='blue',figsize=(10,12))
          plt.figure(figsize=(10,12))
          plt.scatter(df["date"],df["sentiments"],color='blue')
          df = df.sort_values("date")
          plt.plot(df["date"],df["sentiments"],color='red',linestyle='--',linewidth=1)
          plt.show()
```



```
In [16]: plt.savefig('timeline.png') # 看不到？改一改？
```



```
In [2]: import requests
import json

# DeepSeek API 端点
url = "https://api.deepseek.com/v1/chat/completions"

# 替换为您的 DeepSeek API 密钥
API_KEY = "sk-78e664727760463394135e9246255cfa" # 直接复制过来

# 请求头, 包含 API 密钥和内容类型
headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json"
}
```

```
In [3]: # 患者描述文本
text = (
    "我今年58岁, 退休后感到生活失去了重心, 开始出现失眠、头痛和疲乏无力的症状。"
    "后来, 皮肤变得异常敏感, 连衣服的触碰都像针扎一样疼痛。"
    "我常常感到心慌、胸闷, 背部沉重得像压了一块石头。"
    "对光线和声音变得极度敏感, 电话铃声都会让我惊恐。"
    "多次到医院检查, 结果都显示没有器质性疾病。"
    "我变得不愿出门, 不想与人交流, 整天把自己关在屋里, 拉紧窗帘, 感觉生活毫无意义。"
)

# 构建提示词, 要求模型提取细粒度情感实体
prompt = (
    "请从以下患者描述中提取出具体的身体部位、症状以及对应的情感状态, "
    "并以 JSON 格式返回, 格式如下: "
    "{ '实体': [ { '部位': '...', '症状': '...', '情感': '...' }, ... ] }\n\n"
    f"患者描述: {text}"
)
```

```
In [4]: # 请求体, 包含模型参数和提示词
data = {
    "model": "deepseek-chat",
    "messages": [
        { "role": "user", "content": prompt }
    ],
    "temperature": 0.3
}

try:
    # 发送 POST 请求
    response = requests.post(url, headers=headers, data=json.dumps(data))

    # 检查响应状态码
    if response.status_code == 200:
        # 解析 JSON 响应
        result = response.json()
        # 提取模型生成的内容
        generated_text = result['choices'][0]['message']['content']
        print("细粒度情感实体抽取结果:")
```

```
print(f"  错误信息: {response.text}")

except requests.exceptions.RequestException as e:
    # 处理网络请求异常
    print(f"网络请求失败: {e}")
except json.JSONDecodeError as e:
    # 处理 JSON 解析异常
    print(f"JSON 解析失败: {e}")
except Exception as e:
    # 处理其他异常
    print(f"发生未知错误: {e}")
```

```
细粒度情感实体抽取结果:
{
  "实体": [
    {
      "部位": "全身",
      "症状": "失眠",
      "情感": "失去重心"
    },
    {
      "部位": "头部",
      "症状": "头痛",
      "情感": "疲乏无力"
    },
    {
      "部位": "皮肤",
      "症状": "异常敏感, 触碰如针扎疼痛",
      "情感": "不适"
    },
    {
      "部位": "心脏/胸部",
      "症状": "心慌、胸闷",
      "情感": "惊恐"
    },
    {
      "部位": "背部",
      "症状": "沉重如压石头",
      "情感": "压抑"
    },
    {
      "部位": "感官(视觉/听觉)",
      "症状": "对光线和声音极度敏感",
      "情感": "惊恐"
    },
    {
      "部位": "行为/心理",
      "症状": "不愿出门、不与人交流、关屋里拉窗帘",
      "情感": "生活无意义"
    }
  ]
}
```

```
In [2]: import re

with open("背影.txt", "r", encoding="utf-8") as f:
    text = f.read()

# 按照句号、感叹号进行分句（保留标点）
sentences = re.split(r'(?=[。！])', text)
sentences = [s.strip() for s in sentences if s.strip()]
numbered_sentences = [(i + 1, s) for i, s in enumerate(sentences)]
#numbered_sentences = numbered_sentences[0:5] #大模型抽取很慢，提取文本的前n条句子

for num, sent in numbered_sentences:
    print(f"{num}: {sent}")
```

1: 我与父亲不相见已二年余了，我最不能忘记的是他的背影。

2: 那年冬天，祖母死了，父亲的差使也交卸了，正是祸不单行的日子。

3: 我从北京到徐州，打算跟着父亲奔丧回家。

4: 到徐州见着父亲，看见满院狼藉的东西，又想起祖母，不禁簌簌地流下眼泪。

5: 父亲说：“事已如此，不必难过，好在天无绝人之路！”

6: ”

回家变卖典质，父亲还了亏空；又借钱办了丧事。

7: 这些日子，家中光景很是惨淡，一半为了丧事，一半为了父亲赋闲。

8: 丧事完毕，父亲要到南京谋事，我也要回北京念书，我们便同行。

9: 到南京时，有朋友约去游逛，勾留了一日；第二日上午便须渡江到浦口，下午上车北去。

10: 父亲因为事忙，本已说定不送我，叫旅馆里一个熟识的茶房陪我同去。

11: 他再三嘱咐茶房，甚是仔细。

12: 但他终于不放心，怕茶房不妥帖；颇踌躇了一会。

13: 其实我那年已二十岁，北京已来往过两三次，是没有什麼要紧的了。

14: 他踌躇了一会，终于决定还是自己送我去。

15: 我再三劝他不必去；他只说：“不要紧，他们去不好！”

16: ”

我们过了江，进了车站。

17: 我买票，他忙着照看行李。

18: 行李太多了，得向脚夫00行些小费才可过去。

19: 他便又忙着和他们讲价钱。

20: 我那时真是聪明过分，总觉得说话不大漂亮，非自己插嘴不可，但他终于讲定了价钱；就送我上车。

21: 他给我拣定了靠车门的一张椅子；我将他给我做的紫毛大衣铺好座位。

22: 他嘱我路上小心，夜里要警醒些，不要受凉。

23: 又嘱托茶房好好照应我。

24: 我心里暗笑他的迂；他们只认得钱，托他们只是白托！

25: 而且我这样大年纪的人，难道还不能料理自己么？我现在想想，我那时真是太聪明了。

26: 我说道：“爸爸，你走吧。”

27: ”他往车外看了看，说：“我买几个橘子去。”

28: 你就在此地，不要走动。

29: ”我看那边月台的栅栏外有几个卖东西的等着顾客。

30: 走到那边月台，须穿过铁道，须跳下去又爬上去。

31: 父亲是一个胖子，走过去自然要费事些。

32: 我本来要去的，他不肯，只好让他去。

33: 我看见他戴着黑布小帽，穿着黑布大马褂，深青布棉袍，蹒跚03地走到铁道边，慢慢探身下去，尚不大难。

34: 可是 he 穿过铁道，要爬上那边月台，就不容易了。

35: 他用两手攀着上面，两脚再向上缩；他肥胖的身子向左微倾，显出努力的样子。

36: 这时我看见他的背影，我的泪很快地流下来了。

37: 我赶紧拭干了泪。

38: 怕他看见，也怕别人看见。

```
except Exception as e:
    print(f"[异常] 第[num]句处理失败: {e}")
    emotion_results.append((num, sent, "无", 0.0))

In [4]: for item in emotion_results:
        print(f"句子{item[0]}: 情感词: {item[2]}, 情感值: {item[3]}")
```

句子1: 情感词: 思念, 情感值: -0.3

句子2: 情感词: 悲伤, 情感值: -0.8

句子3: 情感词: 悲伤, 情感值: -0.8

句子4: 情感词: 悲伤, 情感值: -0.8

句子5: 情感词: 慰藉, 情感值: 0.3

句子6: 情感词: 沉重, 情感值: -0.8

句子7: 情感词: 悲伤, 情感值: -0.8

句子8: 情感词: 平静, 情感值: 0.0

句子9: 情感词: 平静, 情感值: 0.0

句子10: 情感词: 平静, 情感值: 0.1

句子11: 情感词: 关切, 情感值: 0.6

句子12: 情感词: 担忧, 情感值: -0.5

句子13: 情感词: 淡然, 情感值: 0.1

句子14: 情感词: 犹豫, 情感值: 0.1

句子15: 情感词: 担忧, 情感值: -0.3

句子16: 情感词: 平静, 情感值: 0.0

句子17: 情感词: 中性, 情感值: 0.0

句子18: 情感词: 无奈, 情感值: -0.3

句子19: 情感词: 忙碌, 情感值: 0.0

句子20: 情感词: 懊悔, 情感值: -0.3

句子21: 情感词: 温暖, 情感值: 0.6

句子22: 情感词: 关切, 情感值: 0.7

句子23: 情感词: 温暖, 情感值: 0.7

句子24: 情感词: 轻蔑, 情感值: -0.5

句子25: 情感词: 自嘲, 情感值: -0.3

句子26: 情感词: 不舍, 情感值: -0.3

句子27: 情感词: 温暖, 情感值: 0.6

句子28: 情感词: 关切, 情感值: 0.3

句子29: 情感词: 平淡, 情感值: 0.0

句子30: 情感词: 疲惫, 情感值: -0.3

句子31: 情感词: 无奈, 情感值: -0.3

句子32: 情感词: 无奈, 情感值: -0.3

句子33: 情感词: 感伤, 情感值: -0.3

句子34: 情感词: 艰难, 情感值: -0.3

句子35: 情感词: 努力, 情感值: 0.3

句子36: 情感词: 悲伤, 情感值: -0.8

句子37: 情感词: 克制, 情感值: -0.3

句子38: 情感词: 恐惧, 情感值: -0.8

句子39: 情感词: 欣慰, 情感值: 0.3

句子40: 情感词: 温情, 情感值: 0.3

句子41: 情感词: 关切, 情感值: 0.4

句子42: 情感词: 温暖, 情感值: 0.3

句子43: 情感词: 轻松, 情感值: 0.6

句子44: 情感词: 牵挂, 情感值: 0.3

句子45: 情感词: 失落, 情感值: -0.3



丁香医生知识图谱是丁香园（丁香医生母公司）构建的医疗领域专业知识网络，以「疾病、症状、药品、检查」等实体为节点，以「治疗、关联、禁忌」等为关系，形成结构化的医疗知识体系，主要服务于大众健康科普、在线问诊辅助、智能搜索等场景。

第六讲

实体与关系的结构化

覆盖疾病、症状、药品、检查等核心医疗实体（如你提供的演示图中，包含「高血压」「头痛」「阿司匹林」「血压测量」等节点）。

定义 60 + 类医疗关系（如「疾病 - 症状」「疾病 - 药品（治疗）」「药品 - 禁忌」等），通过人工 + 算法结合的方式维护准确性。

业务融合的四层架构采用「实体层→内容层→概念层→业务主题层」的层级结构，既对接专业医学知识（如疾病分类），也关联用户搜索、点击等行为，实现「专业知识 + 用户需求」的结合（例如将「十二指肠溃疡」关联到「消化内科疾病」概念）。

技术支撑：算法 + 人工质控

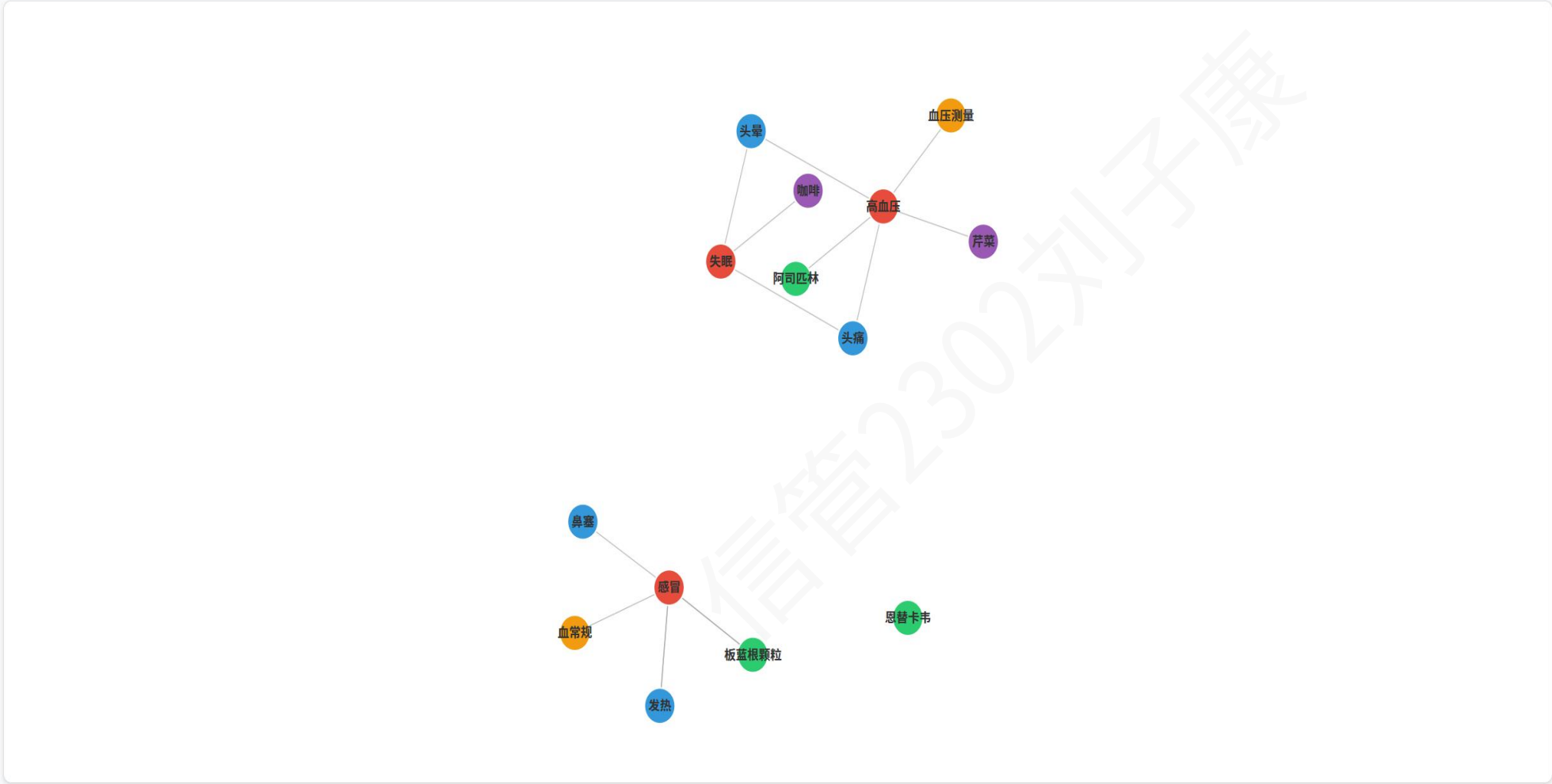
用 SMedBERT（医疗预训练模型）优化实体识别、关系抽取，提升长文本（如病历）的解析能力。

人工审核 + 规则校验确保知识准确率（例如药品禁忌、疾病症状的关联需经专业医生审核）

医疗知识图谱概念演示

此演示基于开源医疗知识图谱项目概念构建，展示了疾病、症状、药品等实体之间的简化关系,真实的医疗知识图谱

实体类型筛选：☒ 疾病 ☒ 症状 ☒ 药品 ☒ 检查 ☒ 食物



图例：■ 疾病 ■ 症状 ■ 药品 ■ 检查 ■ 食物

点击图谱中的节点或连线，详细信息将显示在此处。

应用场景
健康科普与智能搜索
支撑丁香医生「健康百科」的问答式内容（如「高血压吃什么药？」），通过图谱关联疾病、药品、注意事项，让内容更系统。

在线问诊辅助
帮助医生快速获取疾病关联信息（如症状对应的检查项目），同时辅助用户自诊（通过症状匹配可能的疾病）。

内容与服务的闭环
从知识科普（如「头痛的原因」）关联到在线问诊、药品购买等服务，实现「信息→服务」的转化。

核心优势

知识专业性与可信度的「双保障」

医生资源背书：依托丁香园平台覆盖的全国 70%+ 执业医师，知识图谱的实体（如疾病诊断标准）、关系（如「疾病 - 用药」）需经三甲医生审核，避免「网络谣言式」健康信息（例如「感冒用抗生素」这类错误关联会被人工拦截）。

医学数据源支撑：对接《中国药典》《临床诊疗指南》等权威资料，同时整合丁香园「用药助手」的专业数据库（覆盖 1.5 万+ 药品说明书），确保知识的「循证性」（比如「布洛芬的禁忌人群」直接关联药典内容）。

用户场景的「精准适配」

大众友好的知识简化：将专业医学术语（如「上呼吸道感染」）关联到用户常用词（如「感冒」），同时通过知识图谱的「关系链」降低理解门槛（例如「感冒→症状（鼻塞）→缓解方法（生理盐水洗鼻）」，用简单链路替代长篇医学论文）。

行为数据的动态优化：通过用户搜索、点击、问诊数据，调整知识图谱的「关联权重」（例如用户搜索「头痛」时，优先展示「偏头痛」而非「颅内肿瘤」，平衡专业严谨性与大众焦虑缓解）。

业务生态的「深度融合」

问诊场景的效率提升：医生端后台可通过知识图谱快速调取「患者症状→疑似疾病→推荐检查」的链路（例如患者说「腹痛 + 反酸」，图谱自动关联「胃溃疡」及「胃镜检查」建议），缩短问诊时间。

内容 - 服务的闭环衔接：从知识科普（如「高血压饮食禁忌」）直接关联到「在线购药（降压药）」「预约体检（血压监测）」等服务，用图谱的「实体关系」实现「信息→交易」的转化。

核心不足

知识覆盖的「局限性」

病种覆盖偏「常见病」：目前图谱核心覆盖的是感冒、高血压、糖尿病等 1000+ 常见病，罕见病（如「渐冻症」细分类型）、专科疑难病（如「自身免疫性肝病」）的实体、关系完善度较低。

「长尾知识」不足：前沿治疗方案（如某癌症的最新靶向药）、小众人群需求（如「孕妇感冒用药」的细分孕周禁忌）的更新速度慢于专业医学进展（通常滞后 3-6 个月）。

专业深度的「平衡难题」

大众场景下的「信息简化过度」：为了降低理解门槛，部分专业内容被「模糊化」（例如「抗生素的使用指征」仅说明「细菌感染可用」，但未区分「革兰氏阳性 / 阴性菌」的差异），可能无法满足专业用户（如基层医生）的需求。

「非标准化知识」的缺失：医学领域存在大量「个体化诊疗」知识（如「某患者的高血压用药需结合肾功能」），这类依赖临床经验的内容难以被知识图谱的「标准化关系」覆盖。

技术与运营的「成本瓶颈」

人工审核的高成本：知识图谱的实体、关系需持续人工（医生）维护，对于新疾病（如新发传染病）、新药品的更新效率低（例如某新药上市后，需 1-2 个月完成图谱关联）。

算法的「冷启动」局限：对于低搜索量的知识（如「儿童罕见皮肤病的护理」），算法难以通过用户行为优化关联权重，导致这类内容的推荐、检索体验较差。

```

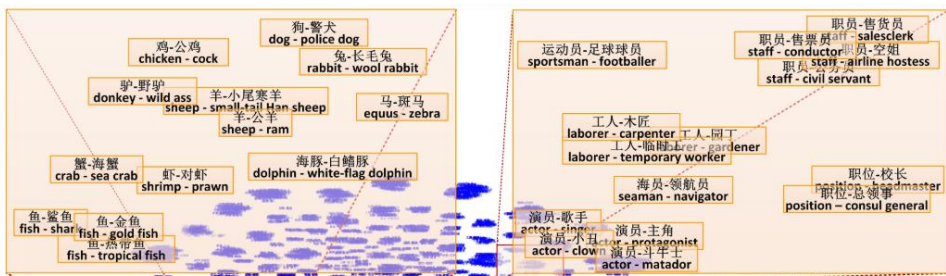
graph LR
    subgraph Search [搜索]
        Entity[实体] --> SE[搜索引擎]
        Website[网页] --> SE
        SE --> SR[搜索结果]
        KB_Crawler[百科爬虫] -.-> SR
    end
    subgraph Selection [候选上位词选取]
        SR --> E[候选上位词选取器]
        KB_Page[百科页面] <--> E
    end
    subgraph Sorting [排序]
        E --> C[候选上位词集]
        KB_Page --> S[上位词排序器]
        C --> S
    end
    S --> O[有序上位词集]
    O --> SE

```

实体上位词抽取过程

系统特点:

- 目前《大词林》2.0版已拥有实体30,102,845 (三千万), 上位词182,079 (十八万), 优质的实体上下位关系对15,577,846 (一千五百万对), 属性-值对79,568,791 (七千九百万对), 关系(属性)数436,961 (四十三万)。



(1040, 'Too many connections')

追溯 开关切换到复制粘贴视图

► 局部变种

```
./discover/views.py在discover_WSDTest
```

```
1806.     if Entity.objects.filter(name=q).count() > 1:
```

► 局部变种

```
/root/miniconda3/envs/bigcilin/lib/python2.7/site-packages/django/db/models/query.py:在count
```

```
294.     return self.query.get_count(using=self.db)
```

► 局部变种

```
/root/miniconda3/envs/bigcilin/lib/python2.7/site-packages/django/db/models/sql/query.py在get_count
```

```
390.         number = obj.get_aggregation(using=using)[None]
```

► 局部变种

```
/root/miniconda3/envs/bigcilin/lib/python2.7/site-packages/django/db/models/sql/query.py:164:Warning: get_aggregation
```

```
356.      result = query.get_compiler(using).execute_sql(SINGLE)
```

► 局部变种

```
/root/miniconda3/envs/bigcilin/lib/python2.7/site-packages/django/db/models/sql/compiler.py in execute_sql
```

```
785.         cursor = self.connection.cursor()
```

► 局部变种

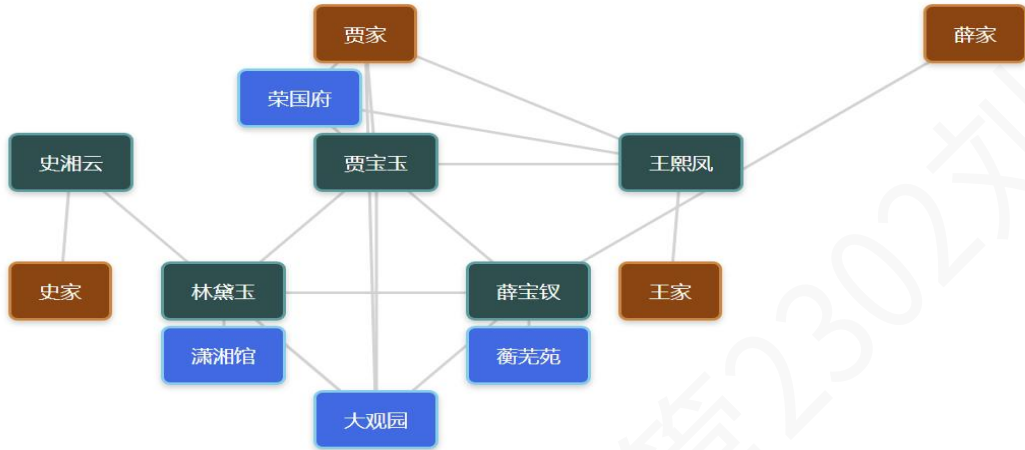
Untitled-1* x

单击标尺上的“+”图标以添加媒体查询

《红楼梦》核心知识图谱

图例说明

- 人物节点
- 家族节点
- 场景节点
- 知识关联线



```
1 <!DOCTYPE html>
2 <html lang="zh-CN">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>红楼梦 知识图谱</title>
7 </head>
8 <body>
9   /* 全局样式重置 */
10   * {
11     margin: 0;
12     padding: 0;
13     box-sizing: border-box;
14     font-family: "微软雅黑", "宋体", Arial, sans-serif;
15   }
16   body {
17     background-color: #f5f5f5;
18     padding: 20px;
```

文件 CC Libraries 插入 CSS 设计器

桌面 管理站点

本地文件 大小

- 桌面
- 此电脑
- 网络
- 桌面项目

DOM 资源 代码片断

- html
- head
- body

《红楼梦》人物关系知识图谱

基于知识图谱技术构建的《红楼梦》人物关系可视化系统，展现贾、王、史、薛四大家族的核心人物与复杂关系[citation:2]

人物搜索

输入人物名称 (如: 贾宝玉)

关系类型筛选

父子

母子

祖孙

夫妻

外祖孙

母女

姐妹

姑侄

父女

爱情

婚姻

主仆

